Projected Functional Dependencies and Dependency Preservation

5 Projecting the Functional Dependencies

The minimal basis from that document is used as F_{min}. The two references to `Region` are temporarily distinguished as `request_region_id` and `server_region_id`, both are stored as `region_id` in their respective tables as mentioned in the FD and Minimal Basis Document

5.1 Projection procedure

For each relation we:

1. Choose a non-empty attribute set $C \subseteq S$.
2. Calculate $\{C\}^+$ using F_{min}.
3. Restrict the closure to S using $\{C\}^+ \cap S$.
4. Record the non-trivial FDs and simplify them to a minimal basis.

In principle every non-empty subset is considered. The first few projections are shown in more detail. The remaining relations use the same procedure:

$$\boxed{\text{Relation} \longrightarrow \text{Closure} \longrightarrow \text{Intersection} \longrightarrow \text{Projected basis}}$$

5.2 Worked projection: Region

$$\begin{aligned} \textbf{Relation:} \quad & R = \{region_id, region_name, country\}, \\ \textbf{Closure:} \quad & \{region_id\}^+ = \{region_id, region_name, country\}, \\ \textbf{Intersection:} \quad & \{region_id\}^+ \cap R = R. \end{aligned}$$

Projected minimal basis:

$$region_id \to region_name, \qquad region_id \to country.$$

5.3 Worked projection: GPUServer

$$\begin{aligned} \textbf{Relation:} \quad GS = \{&machine_id, cpu_capacity, memory_capacity, gpu_capacity, \\ &server_region_id, gpu_type_name\}. \end{aligned}$$

Using F_{min}:

Closure: $\{machine_id\}^+ = \{machine_id, cpu_capacity, memory_capacity, gpu_capacity, server_region_id, gpu_type_name, memory_gb\}$,

Intersection: $\{machine_id\}^+ \cap GS = GS$.

Projected minimal basis:

$machine_id \to cpu_capacity$ $\quad$ $machine_id \to memory_capacity$

$machine_id \to gpu_capacity$ $\quad$ $machine_id \to server_region_id$

$machine_id \to gpu_type_name$.

Note that `server_region_id` is stored as `region_id` in the table.

5.4 Worked projection: MachineMetric

Relation: $MM = \{machine_id, metric_id, start_time, end_time, worker_load, gpu_utilization, cpu_utilization\}$.

First we check each part of the weak entity identifier:

Single-attribute closures: $\{machine_id\}^+ \cap MM = \{machine_id\}$, $\{metric_id\}^+ \cap MM = \{metric_id\}$.

Neither alone determines the metric row. Thus:

Closure: $\{machine_id, metric_id\}^+ = \{machine_id, metric_id, cpu_capacity, memory_capacity, gpu_capacity, server_region_id, gpu_type_name, memory_gb, start_time, end_time, worker_load, gpu_utilization, cpu_utilization\}$,

Intersection: $\{machine_id, metric_id\}^+ \cap MM = MM$.

Projected minimal basis:

$\{machine_id, metric_id\} \to start_time$ $\quad$ $\{machine_id, metric_id\} \to end_time$

$\{machine_id, metric_id\} \to worker_load$ $\quad$ $\{machine_id, metric_id\} \to gpu_utilization$

$\{machine_id, metric_id\} \to cpu_utilization$.

This confirms that both attributes are required in the weak entity's full identifier.

5.5 Worked projection: Hosts

Relation: $H = \{machine_id, model_id\}$,

Closures: $\{machine_id\}^+ \cap H = \{machine_id\}$, $\{model_id\}^+ \cap H = \{model_id\}$,

Intersection: $\{machine_id, model_id\}^+ \cap H = H$.

Projected minimal basis: No non-trivial FDs. The final intersection produces only trivial dependencies.

5.6 Remaining relation projections

The remaining relations follow the same procedure as shown above.

GPUType.

$$\begin{aligned}
\textbf{Relation:} \quad & G = \{gpu_type_name\}, \\
\textbf{Closure:} \quad & \{gpu_type_name\}^+ = \{gpu_type_name, memory_gb\}, \\
\textbf{Intersection:} \quad & \{gpu_type_name\}^+ \cap G = G.
\end{aligned}$$

Projected minimal basis: No non-trivial FDs.

SpecificGPUType.

$$\begin{aligned}
\textbf{Relation:} \quad & SG = \{gpu_type_name, memory_gb\}, \\
\textbf{Closure:} \quad & \{gpu_type_name\}^+ = \{gpu_type_name, memory_gb\}, \\
\textbf{Intersection:} \quad & \{gpu_type_name\}^+ \cap SG = SG.
\end{aligned}$$

Projected minimal basis:

$$gpu_type_name \to memory_gb.$$

AIModel.

$$\begin{aligned}
\textbf{Relation:} \quad & M = \{model_id, model_name\}, \\
\textbf{Closure:} \quad & \{model_id\}^+ = \{model_id, model_name\}, \\
\textbf{Intersection:} \quad & \{model_id\}^+ \cap M = M.
\end{aligned}$$

Projected minimal basis:

$$model_id \to model_name.$$

InferenceRequest.

$$\begin{aligned}
\textbf{Relation:} \quad Q = \{&request_id, submitted_at, input_size, status, \\
&response_time_ms, request_region_id, model_id, machine_id\}.
\end{aligned}$$

The closure of `request_id` also reaches attributes outside Q through `model_id` and `machine_id`; only attributes in Q are kept during projection.

$$\begin{aligned}
\textbf{Closure:} \quad & \{request_id\}^+ \\
& = \{request_id, submitted_at, input_size, status, response_time_ms, \\
& \quad request_region_id, model_id, machine_id, model_name, \\
& \quad cpu_capacity, memory_capacity, gpu_capacity, server_region_id, \\
& \quad gpu_type_name, memory_gb\}, \\
\textbf{Intersection:} \quad & \{request_id\}^+ \cap Q = Q.
\end{aligned}$$

Projected minimal basis:

$request_id \to submitted_at$ $\quad$ $request_id \to input_size$

$request_id \to status$ $\quad$ $request_id \to response_time_ms$

$request_id \to request_region_id$ $\quad$ $request_id \to model_id$

$request_id \to machine_id.$

`request_region_id` is stored as `region_id` in the table as mentioned above.

CompatibleWith.

$$\begin{aligned} \textbf{Relation:} \quad & CW = \{model_id, gpu_type_name\}, \\ \textbf{Closures:} \quad & \{model_id\}^+ \cap CW = \{model_id\}, \\ & \{gpu_type_name\}^+ \cap CW = \{gpu_type_name\}, \\ \textbf{Intersection:} \quad & \{model_id, gpu_type_name\}^+ \cap CW = CW. \end{aligned}$$

Projected minimal basis: No non-trivial FDs.

5.7 Projected Minimal Bases Summary

Relation	Projected minimal basis
Region	$region_id \to region_name$, $region_id \to country$
GPUType	No non-trivial FDs
SpecificGPUType	$gpu_type_name \to memory_gb$
AIModel	$model_id \to model_name$
GPUServer	$machine_id \to$ each non-key server attribute
InferenceRequest	$request_id \to$ each non-key/FK request attribute
MachineMetric	$\{machine_id, metric_id\} \to$ each metric attribute
Hosts	No non-trivial FDs
CompatibleWith	No non-trivial FDs

6 Dependency Preservation

Let P be the union of the projected minimal bases above. After mapping the temporary region role names back to the corresponding local `region_id` columns, P contains the same 21 non-trivial dependencies as F_{min}:

$$P = F_{min} \qquad \Longrightarrow \qquad P^+ = F_{min}^+.$$

Every dependency can therefore be checked within a single final relation. No dependency from F_{min} requires joining relations in order to enforce it, so the current decomposition is **dependency preserving** for the identified functional dependencies.

7 Conclusion

The minimal basis was projected onto every relation using the lecture procedure of calculating closures, intersecting with the target relation, and simplifying the result. The union of the projected bases is equivalent to F_{min}, so the updated Sprint Two schema preserves the identified functional dependencies.